SILAFIT · PARTNER INTEGRATION

Partner API reference

How a partner website gives its leads a SilaFit account with PRO, and later ends it. Covers the two endpoints, authentication, and the end-to-end encryption of every payload.

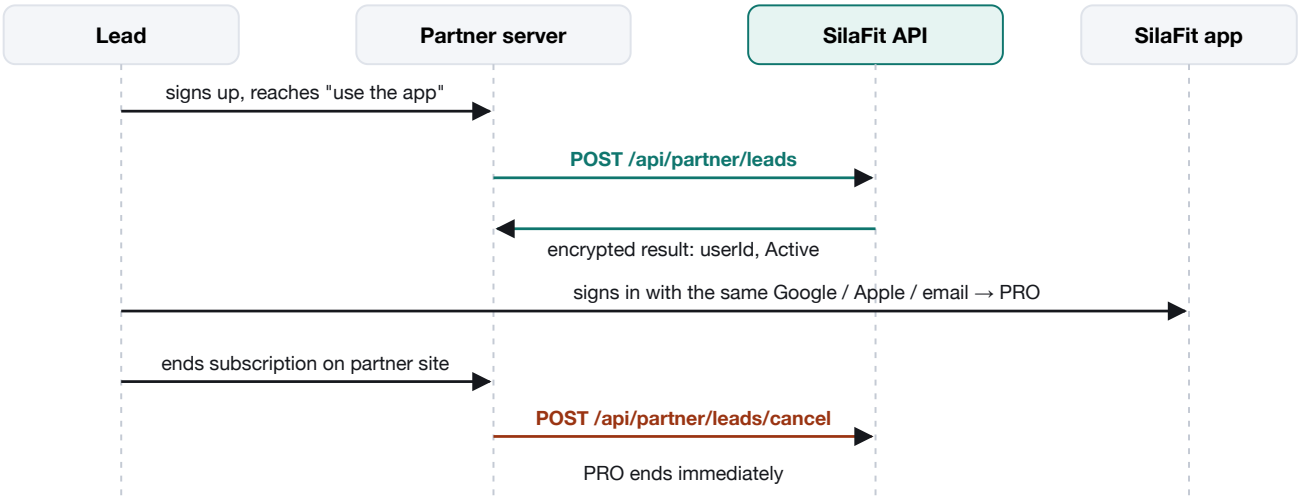
Version	Date	Base URL	Audience
1.0	22 October 2026	https://api.sila.fitness	Partner server developers

Contents

1. How it works
2. What you receive from SilaFit
3. Authentication
4. Payload encryption
5. Provision a lead
6. Cancel a lead
7. Responses and errors
8. Freshness, replay and retries
9. How the lead reaches the account in the app
10. Reference implementation (Node.js)
11. Test vector
12. Go-live checklist

1 How it works

A lead signs up on the partner website with Apple, Google or email. When they reach the "use the app" screen, the partner's **server** calls SilaFit to provision them. SilaFit creates the account, or reuses an existing account with the same email, and grants PRO with no expiry date. PRO stays active until the partner calls cancel, which ends it immediately.



> **Server to server only**
> Never call these endpoints from a browser or a mobile app. The Basic credentials and the payload key must stay on the partner's server.

2 What you receive from SilaFit

ITEM	USE
Username and password	HTTP Basic authentication on every request (section 3).
Payload key	32 random bytes, sent as base64. Used to encrypt requests and decrypt responses (section 4).

Both are delivered out of band (never by plain email) and can be rotated on request. Treat the key like a password: anyone holding it can create PRO accounts.

3 Authentication

Every request carries standard HTTP Basic credentials:

```
Authorization: Basic base64(username + ":" + password)
```

A missing or wrong header is rejected with `401` and `WWW-Authenticate: Basic realm="SilaFit Partner"` before the body is read. Basic auth proves *who* is calling. The payload encryption in section 4 protects *what* is sent, so lead details stay confidential and tamper-proof even inside TLS-terminating infrastructure.

Requests are limited to **60 per minute per client IP**; above that the API returns `429`.

4 Payload encryption

Request bodies and successful response bodies are sealed with **AES-256-GCM**, an authenticated cipher: it keeps the data secret and also detects any change to it.

PARAMETER	VALUE
Algorithm	AES-256-GCM
Key	The 32-byte payload key (decode it from base64).
Nonce (IV)	12 random bytes from a secure random generator, **new for every message**. Never reuse a nonce with the same key.
Authentication tag	16 bytes (128 bits).
Associated data (AAD)	UTF-8 string `silafit-partner:` followed by the purpose (table below). It is authenticated but not sent.
Plaintext	UTF-8 JSON object.
Encoding	base64url without padding. Standard base64 is also accepted on requests.

Wire format

The encrypted string is the three parts joined in this order, then base64url-encoded:



Many libraries (Node.js, .NET, Go) return the tag separately; append it after the ciphertext. Others (Python `cryptography`, Java `AES/GCM/NoPadding`) already append it, so add only the nonce in front.

Purposes

The purpose ties each payload to one endpoint and one direction. A payload sealed for one purpose fails authentication under any other, so a provision request can't be replayed as a cancel, and a response can't be passed off as a request.

ENDPOINT	REQUEST AAD	RESPONSE AAD
Provision	`silafit-partner:provision`	`silafit-partner:provision-response`
Cancel	`silafit-partner:cancel`	`silafit-partner:cancel-response`

To encrypt a request

1. Build the JSON payload, with `issuedAtUtc` set to now.
2. Generate a 12-byte random nonce.
3. Encrypt the UTF-8 JSON with the key, nonce and request AAD.
4. Concatenate nonce ‖ ciphertext ‖ tag; base64url-encode.
5. Send `{ "payload": "<that string>" }`.

To decrypt a response

1. Take `data.payload` from the response and base64url-decode it.
2. Split: first 12 bytes nonce, last 16 bytes tag, the rest ciphertext.
3. Decrypt with the key, nonce, tag and response AAD.
4. Parse the UTF-8 JSON.
5. Check `requestIssuedAtUtc` equals the `issuedAtUtc` you sent.

5 Provision a lead

POST `/api/partner/leads`

Call this when the lead reaches the "use the app" screen. SilaFit creates the account (or reuses the one with this email) and grants PRO until you cancel. Calling it again for the same lead is safe, and calling it after a cancel turns PRO back on.

HTTP request

```
POST /api/partner/leads HTTP/1.1
Host: api.sila.fitness
Authorization: Basic cGFydG5lcjpzZWNyZXQ=
Content-Type: application/json

{ "payload": "AAECAwQFBgcICQoLPCCmereRrH7_FOTuw6AcT7n0…" }
```

Payload (before encryption) · AAD `silafit-partner:provision`

```
{
  "partnerUserId": "lead-123",
  "email": "ana@example.com",
  "displayName": "Ana",
  "provider": "Google",
  "issuedAtUtc": "2026-10-22T10:15:30.123Z"
}
```

FIELD	REQUIRED	RULES
`partnerUserId`	Yes	Your own stable id for the lead, at most 100 characters. Cancel uses the same id.
`email`	Yes	A valid address, at most 256 characters. The app finds the account by this email (section 9).
`displayName`	No	Trimmed, cut to 100 characters. Only used when a new account is created.
`provider`	Yes	`Apple`, `Google` or `Email`: how the lead signed up with you (case-insensitive).
`issuedAtUtc`	Yes	ISO 8601 timestamp of when you built this request. Must be within 5 minutes of SilaFit's clock.

Result (after decrypting) · AAD `silafit-partner:provision-response`

```
{
  "partnerUserId": "lead-123",
  "userId": "0286906e-097f-49aa-85d0-cf76b2d49398",
  "status": "Active",
  "isNewAccount": true,
  "proGranted": true,
  "requestIssuedAtUtc": "2026-10-22T10:15:30.123Z"
}
```

FIELD	MEANING
`userId`	The SilaFit account behind the lead. Store it for support cases.
`status`	Always `Active` on a successful provision.
`isNewAccount`	`false` when an account with this email already existed and the lead was attached to it.
`proGranted`	`false` when that account already has its own paid App Store or Google Play subscription. It stays in place and the user keeps PRO through it; cancelling the lead never ends a plan the user pays for themselves.
`requestIssuedAtUtc`	Your request's `issuedAtUtc`, to the millisecond. Match it to detect replayed responses.

6 Cancel a lead

`POST` `/api/partner/leads/cancel`

Call this when the lead ends their subscription on your website. PRO ends **immediately**; the account itself stays, and the user keeps their data on the free tier.

Payload (before encryption) · AAD `silafit-partner:cancel`

```
{ "partnerUserId": "lead-123", "issuedAtUtc": "2026-10-22T11:00:00.000Z" }
```

Result (after decrypting) · AAD `silafit-partner:cancel-response`

```
{
  "partnerUserId": "lead-123",
  "userId": "0286906e-097f-49aa-85d0-cf76b2d49398",
  "status": "Cancelled",
  "isNewAccount": false,
  "proGranted": false,
  "requestIssuedAtUtc": "2026-10-22T11:00:00.000Z"
}
```

Cancelling a lead that is already cancelled succeeds again with the same result.

7 Responses and errors

Every response is JSON in the SilaFit envelope. On success, `data.payload` is the sealed result:

```
{ "success": true, "message": null, "data": { "payload": "<encrypted result>" } }
```

Errors are **not** encrypted, so you can still read them when your key or format is wrong. They hold only a short `message` for your developers and never any lead data:

```
{ "success": false, "message": "provider must be Apple, Google or Email.", "data": null }
```

STATUS	MEANING	WHAT TO DO
200	Applied.	Decrypt `data.payload` and check `requestIssuedAtUtc`.
400	The payload decrypted, but the JSON or a field is invalid.	Fix the field named in `message`. Don't retry unchanged.
401	Wrong Basic credentials; a payload that won't decrypt (wrong key, AAD, nonce/tag layout or encoding); or `issuedAtUtc` more than 5 minutes off.	Check credentials, key and purpose against the test vector (section 11). Check your server clock (use NTP).
404	Cancel for a `partnerUserId` that was never provisioned.	Check the id; nothing to cancel.
409	The request is older than one already applied for this lead; the email belongs to a different lead; or the SilaFit account with this email is deactivated.	For a stale request, nothing to do: a newer one already won. For the other two, contact SilaFit support.
429	Rate limit exceeded.	Back off and retry with a new payload.
5xx	SilaFit-side failure.	Retry with exponential backoff and a new payload.

8 Freshness, replay and retries

- **Freshness:** `issuedAtUtc` must be within 5 minutes of SilaFit's clock, either way. A captured request stops working after that.

- **Ordering:** SilaFit stores the newest `issuedAtUtc` it has applied for each lead and rejects anything older with `409`. A provision delayed in a queue can never undo a cancel sent after it.
- **Retries:** both endpoints are idempotent. To retry, build a new payload with a new `issuedAtUtc` and a new nonce; don't resend the old one.
- **Responses:** reject any response whose `requestIssuedAtUtc` doesn't match the request you just sent.

9 How the lead reaches the account in the app

The account is found by **email**, so always send the email the lead will use in the SilaFit app.

SIGNED UP WITH	IN THE SILAFIT APP
Google	"Continue with Google" with the same Google account opens the provisioned account with PRO.
Apple	"Sign in with Apple" works the same way, as long as Apple shares the same email with SilaFit. If the lead chose *Hide My Email*, Apple gives each developer team a different relay address, so it won't match unless your website and SilaFit share an Apple developer team. In that case, collect the real email or point the lead to email registration.
Email	"Register with email" with the same address. The lead gets a verification code, and the password they choose is set on the provisioned account.

> **Tip for the "use the app" screen**
> Tell the lead to sign in with the same method and email they just used on your website.

10 Reference implementation (Node.js)

Node.js 18 or later, no dependencies. This code has been tested against the SilaFit server implementation.

```
import crypto from "node:crypto";

const key = Buffer.from(process.env.SILAFIT_PAYLOAD_KEY, "base64"); // 32 bytes
const auth = "Basic " +
Buffer.from(`${process.env.SILAFIT_USER}:${process.env.SILAFIT_PASS}`).toString("base64");

function encrypt(obj, purpose) {
  const nonce = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv("aes-256-gcm", key, nonce);
  cipher.setAAD(Buffer.from(`silafit-partner:${purpose}`, "utf8"));
  const ciphertext = Buffer.concat([cipher.update(JSON.stringify(obj), "utf8"),
cipher.final()]);
  return Buffer.concat([nonce, ciphertext, cipher.getAuthTag()]).toString("base64url");
}

function decrypt(payload, purpose) {
  const bytes = Buffer.from(payload, "base64url");
  const decipher = crypto.createDecipheriv("aes-256-gcm", key, bytes.subarray(0, 12));
  decipher.setAAD(Buffer.from(`silafit-partner:${purpose}`, "utf8"));
  decipher.setAuthTag(bytes.subarray(bytes.length - 16));
  const plaintext = Buffer.concat([decipher.update(bytes.subarray(12, bytes.length - 16)),
decipher.final()]);
  return JSON.parse(plaintext.toString("utf8"));
}

async function call(path, purpose, body) {
  const issuedAtUtc = new Date().toISOString();
  const res = await fetch(`https://api.sila.fitness/api/partner/${path}`, {
    method: "POST",
    headers: { "Content-Type": "application/json", Authorization: auth },
    body: JSON.stringify({ payload: encrypt({ ...body, issuedAtUtc }, purpose) }),
  });
  const envelope = await res.json();
  if (!res.ok) return { status: res.status, error: envelope.message }; // errors are not
encrypted

  const result = decrypt(envelope.data.payload, `${purpose}-response`);
  if (new Date(result.requestIssuedAtUtc).getTime() !== new Date(issuedAtUtc).getTime())
    throw new Error("Response does not belong to this request");
  return { status: res.status, result };
}

// The lead reached the "use the app" screen
await call("leads", "provision",
  { partnerUserId: "lead-123", email: "ana@example.com", displayName: "Ana", provider:
"Google" });

// The lead ended their subscription on the website
await call("leads/cancel", "cancel", { partnerUserId: "lead-123" });
```

11 Test vector

Use this to check your encryption before calling the API. With a **fixed** nonce (for testing only; production must use a random nonce every time), your implementation must produce exactly this payload, and must decrypt it back to exactly this plaintext.

INPUT	VALUE
Key (base64)	`AAECAwQFBgcICQoLDA0ODxAREhMUFRYXGBkaGxwdHh8=` bytes 0x00, 0x01, … 0x1f
Nonce (hex)	`000102030405060708090a0b`
AAD	`silafit-partner:provision`
Plaintext	`{"partnerUserId":"lead-123","email":"ana@example.com","displayName":"Ana","provider":"Google","issuedAtUtc":"2026-10-22T10:15:30.123Z"}`
Expected payload	`AAECAwQFBgcICQoLPCCmereRrH7_FOTuw6AcT7n061GRH3JNClTHqT8MbdNofIzGjaB8-TTBB4zl90RdwDoP4Hj6gb5W5Fp1eZq7j51Z5EDxkEgAPniIHp3geIFc77tATEEUB5Ka8qla3JvkuIssg8AAvE_qfxyIakcRTYxKU5vO9xfOfM9t_LfcXT6VYzL17vW20m9tzmqSlP3M83zqgIxYYg`

The plaintext must be serialized exactly as shown (no spaces) to reproduce the payload byte for byte. The API itself accepts any valid JSON formatting.

12 Go-live checklist

- Credentials and payload key stored in your server's secret store, never in client code or logs.
- Your encryption reproduces the test vector in section 11.
- A fresh random nonce and a fresh `issuedAtUtc` for every request, including retries.
- Server clock synced with NTP.
- Responses decrypted with the `-response` purpose, and `requestIssuedAtUtc` checked.
- Provision called when the lead reaches the "use the app" screen; cancel called the moment their subscription ends.
- The returned `userId` stored next to your lead, for support.
- 400, 401 and 409 responses logged with their `message` and alerted on; 429 and 5xx retried with backoff.